Finite Difference Time Domain Basics

John Nehls

5/23/14

Notes: The contents are not needed large number of sections, but are nice to have for planning purposes. Also, '(later)' is the flag for something not completed making searching for and counting the number of unfinnished parts easier.

Contents

1 Introduction **2**

2 Short C Programming Introduction (needed?) **2**
2.1 Main . . . 2
2.2 Pre-processor Funcitons: Include and Define . . . 2
2.2.1 Include . . . 2
2.2.2 Define . . . 2
2.3 Variables, For Loops, and File Output . . . 3
2.4 I/O File . . . 3
2.5 Plotting in Python . . . 3

3 Introduction to the FDTD Method **3**
3.1 Yee Scheme . . . 3
3.2 FD Stencil and Code . . . 3

4 BareBones Simulation **4**
4.1 Simple Boundary Conditions:PMC and PEC . . . 4
4.2 Sources . . . 5

5 Terminating the Grid (good name? (later)) **5**
5.1 Simple Absorbing Boundary Conditions . . . 6

6 Total Field/Scattered Field **6**

7 Inhomogeneous Medium **7**

8 Lossy Material **7**

Appendices **8**

A Plotting in Python **8**

B Complete Codes **11**
B.1 Additive Source . . . 11
B.2 ABC . . . 12
B.3 TFSF . . . 13
B.4 Inhomogeneous Medium . . . 14
B.5 Lossy Material . . . 15
B.5.1 Simple Lossy . . . 15
B.5.2 Impedance Matched . . . 17

1 Introduction

This document will serve as an introduction to parrellel computing using the Finite Time Difference Time Domain (FDTD) as our example and C as our programming language. (more later)

Our goa ls for this chapter are to:

i. Learn the basics of C programming.

ii. Understand a quick overview of the FDTD method.

iii. Merge the first two goals and create instructive FDTD pograms.

2 Short C Programming Introduction (needed?)

For our first programs, we will be most concered with implementing our programs simply, spending the as little energy on proper programming practices as possible. With this in mind, some programming instruction is needed to accomplish our goals.

2.1 Main

Shortest program:

```
int main(){
  /*comment: code goes here.*/
  return 0;
}
```

the int main() function is required for every program. It is the function that the computer finds and follows when the code is run. Thus, the code you want to run must either be in or be called inside of the main function. The int before main indicates that the function returns a an integer, which it does, default, at the end of the function a 'return 0;' line is put in for you if you do not explicitly specify a return. 0 indicates that the program ran succesfully.

j

2.2 Pre-processor Funcitons: Include and Define

When a C-Program is compiled, the pre-processor first looks through the program for the lines start with '# '. and completes a specified actions before the rest of the program is analyzed. Here, we are particularly concerned with two pre-processor actions: include and define, both of which are usually done fist in a program. The details of the actions are not important to us and my be found (cite). We will give a short introduction to the actions and use them like a black box.

2.2.1 Include

An include line looks like the following (need to center)(later,

```
#include <stdio.h>
```

The simple understanding this action is that the compiler finds the stdio.h text document, which we already have on our computer, and pastes it to the top of the file. This enables us to use funtions written stdio.h in the rest of the program, saving us countless hours writting functions that others have written and placed in libraries. As specific example, stdio.h contains function defintions for file in and output operations (cite link).

2.2.2 Define

Her is a define statement,

```
#define N 200
```

when the compiler sees this line, it searches through the rest of the code and replaces each instance of N with 200.

(adress later) The reason this is done rather than simply stating 'int N = 200;' within the main function, is that line 12 would fail and resultin an error. Try it. When intitilizing the arrays.. (later)

2.3 Variables, For Loops, and File Output

Moving on to higherlevel, less c specific practices, since you know what variables and for loops are, we will handle them all in a simple example

```
double imp0 = 377.0;
int maxT = 450;
int n, curT;
double ez[200] = {0.0} , hy [200] = {0.0};

for ( curT = 0; curT < maxT; curT++){

}
```

Spending too much time on this... I will finish this all (later).

2.4 I/O File

Include barebones code 1, 3, 5, 10-18, 39-42. This will explain writing to files (important?) (later) fopen and fprintf...

```
FILE *fp;
fp=fopen(``data.txt'',''w'');
/* output paramters to the first line */
fprintf(fp, ``%d %s %d %s %f %c ``, N, `` ``,  maxT,  `` ``,  imp0, '\n\
');
int fclose(fp);
```

2.5 Plotting in Python

To handle the plotting, we will use the synatacticly simple Python programing language. This is accomplished by having each of our programs output its data to a text file. Then, after the work-horse C program is finished, a simple python script may be run that reads in the text file and displays an electric field plot. The Python script and details about it may be found in the appendix of this document (cite and link).

3 Introduction to the FDTD Method

Maxwell's equations..The Finite Difference Time Domain Method... (later)

3.1 Yee Scheme

We will ve using the Yee Scheme ... (later)

Miro's BPM course material could be a good source for this information.

3.2 FD Stencil and Code

- Miro's BPM course material could be a good source for this backgroundinformation.
- Second order derivative approximation
- describe the FD stencil
- describe the online FD code won't have to do it in later section.
- Courant number and limit (for now operate in limit)
- Relative Permitivity/Permiability

Need to spend a fair amount of time explaining these two lines:

```
hy[n] += (ez[n+1] - ez[n]) / imp0;

ez[n] += (hy[n] - hy[n-1]) * imp0;
```

4 BareBones Simulation

Our first code is fairly simple and very important. It will serve as the backbone for all of our program in this chapter. Given its importance, we will go through it in its entirity. Stick in there, once this code is understood, it is downhill code wise. With only small changes and/or additions to the code, we learn and see the effects of many FDTD basics.

Here it is in its entirety, take a look over it and/copy it into your favorite text editor. The basics of the primary calculations in this code, lines 27 and 33, are described in the FDTD section above (cite). Much of the rest of the code is simple in its function, but possibly confusing due to the programming syntax. This is why most if it was covered in detail, mainy times verbatim, in the introductory C-programming section above (cite).

// (may I assume that the comments do enough of a job explaining what the code does) If not exlain, you guessed it (later)

```
1  #include <stdio.h>              /* include in/out functions */
2  #include <math.h>               /* include math functions -- need lm flags while compiling*/
3  #define  N 200                  /* define size of ez and hy arrays */
4
5  int main(){
6
7    double imp0 = 377.0;          /* impedence of freespace */
8    int maxT = 250;               /* total simulation time */
9    int n, curT;                  /* declare here since variables cannot be defined in loop
       declairations*/
10
11   /* initialize grids */
12   double ez[N] = {0.0} , hy [N] = {0.0};
13
14   /* for outputting data to a file */
15   FILE *fp;
16   fp=fopen("data.txt","w");
17   /* output paramters to the first line */
18   fprintf(fp, "%d %s %d %s %f %c ", N, " ",  maxT,  " ",  imp0, '\n');
19
20
21   /* Step in time-- main loop */
22   for ( curT = 0; curT < maxT; curT++){
23
24     /* Magnetic Field */
25     /* update H-field -- doesn't evaluate last node, it remains 0 (PMC)*/
26     for (n = 0; n < N-1; n++){
27       hy[n] += (ez[n+1] - ez[n]) / imp0;
28     }
29
30     /* Electric Field */
31     /* update E-field -- doesn't evaluate first node, it remains 0 (PEC)*/
32     for (n = 1; n < N; n++){
33       ez[n] += (hy[n] - hy[n-1]) * imp0;
34     }
35
36     /* hardwire a source node */
37     ez[0] = exp(-(curT - 30.0) * (curT - 30.0) / 100.0);
38
39     /* write data to file*/
40     for (n = 0; n < N; n++){
41       fprintf(fp, "%d %s %f %s %f %c", curT, " ", ez[n], " ", hy[n], '\n');
42     }
43   }
44   /* close the writer */
45   int fclose(fp);
46   return 0;
47 }
```

(provid overview of what the code does, comments enough?

4.1 Simple Boundary Conditions:PMC and PEC

As shown on lines 26 and 32, we simply don't calculate the field at a node, electric or magnetic, when it does not have a the neighbors needed in the FD stencil (cite and link). There isn't an ez node the the 'right' of hy[N-1] (ez[N]) nor an hy node

to the 'left' of ez[0] (create figure and cite). With out knowing it, are implimenting simple boundary conditions: Perfect Magnetic Conductor (PMC) and Perfect Electric Conductor.

(Explain both BCs and how the reflection is achieved?)
We will talk about PEC since it is a common occurence in electro-magnetism... (seen before, mirror, changes sign of electric field to change sign of electric field, no energy lost) Due to the FD derivative
PMC... (inverts sign of magnetic field, changing prop direction (pointing vector))

4.2 Sources

Since we are conserned with modeling real events and not a trivial solution, a source or purtibation must be introduced into the field. Line 37 shows our first source, a simple hardwire source. Here we pick an electic field node (ez[0]) and give change it as a function of time. This is a simple start that allow us to qualitativly check stability and boundary conditions, but it has a major issue that we need to be aware of, the field cannot propagate through the source node. It is 'hardwired' and not calculated using the stencil.

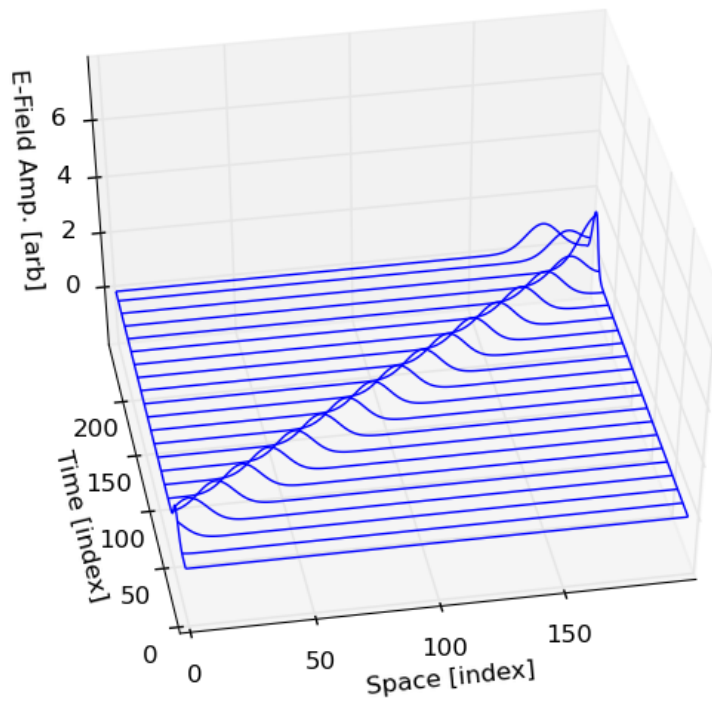

Figure 1: Plot of the barebones code with an hardwired source.

To remidy the propagation issue, rather than hardwire a source, dictating its value, we will only add field amplitude to a node, keep the source nodes dependence on its neighbors. Here we will choose the ez node with index 50 to be the source node. To make the appropriate change to the code, only change lines 36 and 37 to:

```
/* use additive source at node 50 */
ez[50] += exp(-(qTime - 30.) * (qTime - 30.) / 100.);
```

5 Terminating the Grid (good name? (later))

Many times in a simulation, we want to model an interaction in infinite space. So far we have created a a code that implements two differeent reflecting boundary conditions, essentially modeling a sort of resonator. This is the opposite effect of what we need for a simulation in infinite space where, when a field reaches the end of the computational grid, we need it let it

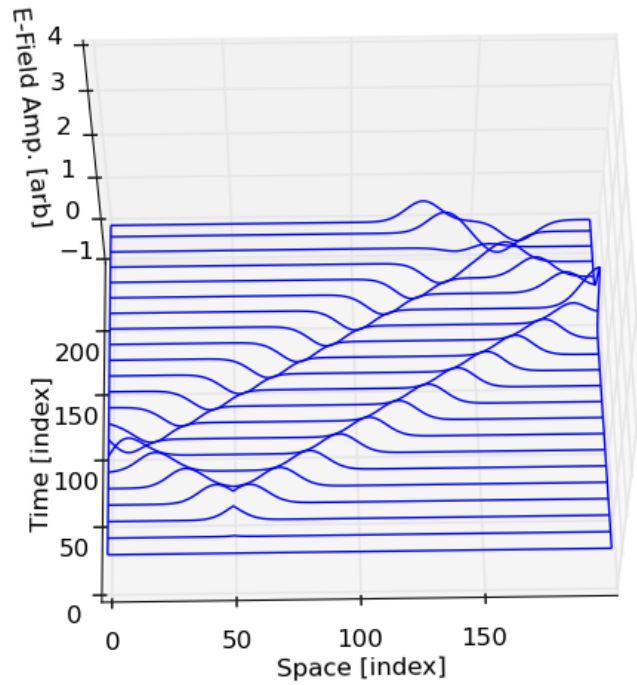

Figure 2: Plot of the barebones code with an additive source. Note that the reflection off of the PMC onthe right side retains the sign of the E-field, where the reflection off of the PEC on the left side flips it.

propagate off to infinitity uninhibited, i.e., we need to it to terminate and produce as little of backscatter as possible. Rather than reflecting boundary conditions, we need absorbing boundary conditions.

5.1 Simple Absorbing Boundary Conditions

To the terminate the field at the boundaries hy[N-1] and ez[0], we will us the FD stencil, in the Courant limit (cite and link), to our advantage. As a reminder, in the Courant limit the distance the field travels in one time step is the grid spaceing. Also, to be complete, we must say that concerned with sources on the grid, where the boundaries nodes may only be effected their inner neighbors. Thus, at each time step ez[0] is updated to be the value of ez[1] at the previous time step and similarly hy[N-1] becomes the value of hy[N-2].
We exploit this by setting each boundary and its innner neighbor equal, and thus its derivative and contribution in the FD stencil to zero. This is achieved adding to line 29

```
ez[0] = ez[1];
```

and adding to line 23

```
hy[N-1] = hy[N-2];
```

6 Total Field/Scattered Field

Give simple understanding or fully explain it?
Simple explination:

- define boundary between and ez and hy node
- one side total field, otherside the source is subtracted
- explained very well in FDTD book

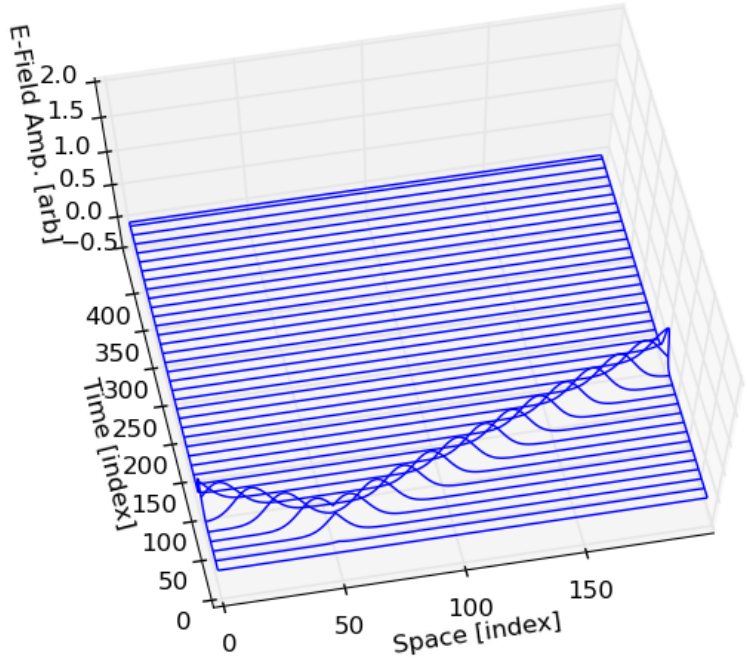

Figure 3: Plot of the barebones code with an additve source and SABC.

- in our case of having a source on the interface of the boundary, TFSF correction dictates the analyitcal form of simple uni-directional wave (as we would expect if TFSF is correct).

7 Inhomogeneous Medium

So far we have only considered a homogeneous medium where the relative permitivity and permiability in (FDTD eqn) remain constant and unity over the grid. Modeling an inhomogeneous grid comes down to creating an array that holds the relative permitvity at each ez node (epsR) and an array that holds the relative permiability at each hy node (muR), and incorporating them into the update calculations on lines 27 and 33 of the barebones code.

8 Lossy Material

(hold until I talk to Jerry (later))

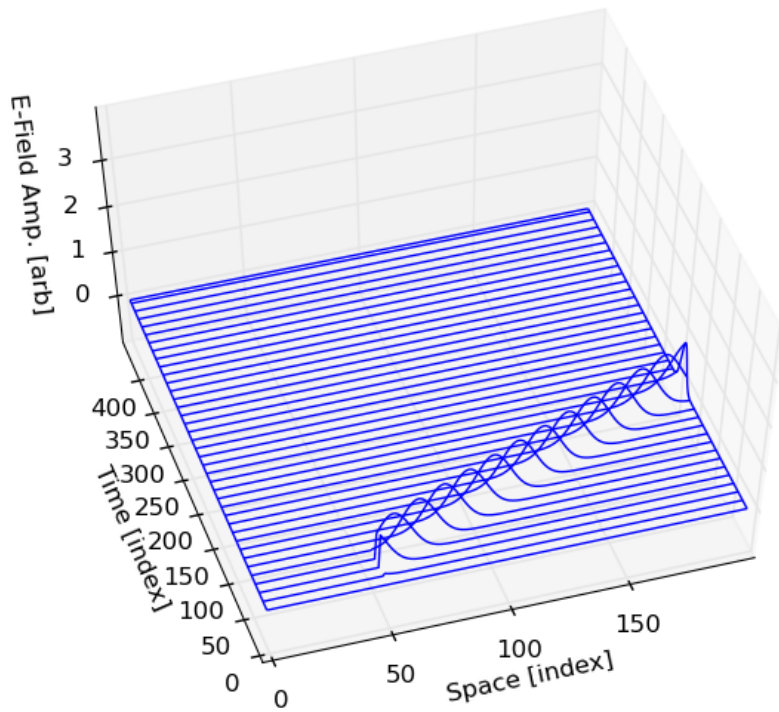

Figure 4: Plot of the barebones code with the TFSF boundary and additive source. off ofthe PMCdoes not change the sign of theE-field.

Appendices

A Plotting in Python

Section Priorities:

- Explain how to use the code (for the audience that will use the code as a blackbox.
- Explain the plots– what waterfall plots are.
- Show the code.

Python is one of the simpliest.. (later)
Brainstorm:

- Explain use with ipython– making it most like Matlab
- I do not want to teach everyone how to install ipython on each operating system, or even python for that matter!
- Maybe I will wait until the end of project the to explain this simple code when I know more about the audience.

```
from pylab import *
from mpl_toolkits.mplot3d import Axes3D

#load the data
rawData = loadtxt("data.txt")
params  = rawData[0,:]
data    = rawData[1:,:]

```

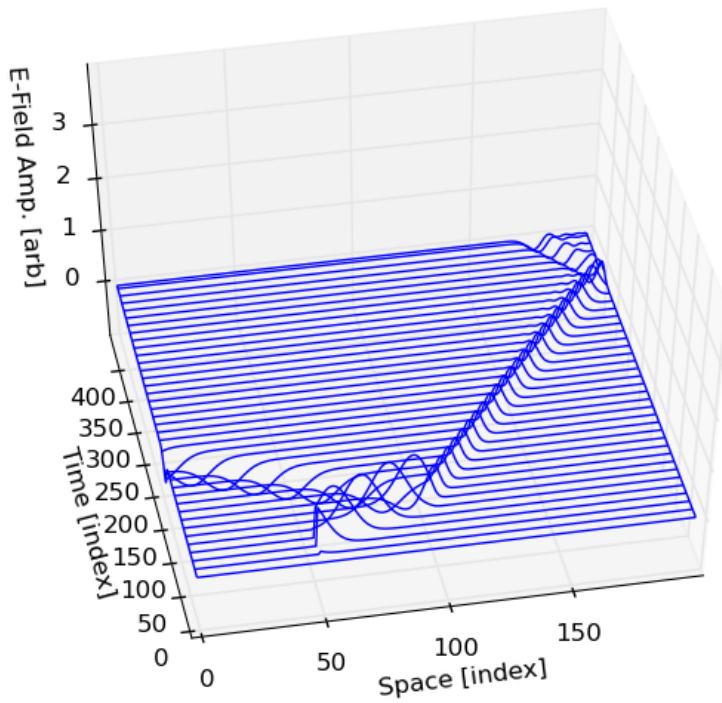

Figure 5: Plot of code with additive source at ez[50], TFSF boundary between hy[49] and ez[50], and dielectic boundary at ez[100]. (explain the reflection and transmittence at boundaries) (later)

```
#Imporant values were storedin the first line
N    = int(params[0])
maxT = int(params[1])
imp0 = params[2]

#fill arrays with the data fromthe text file
t= range(maxT)
ez= zeros((N,maxT),double)
hy= zeros((N,maxT))

for i in t:
    ez[:,i] = data[N*i:N*(i+1),1]
    hy[:,i] = data[N*i:N*(i+1),2]

#makes the script display the plot
show()

#make the plot (need not know the details)
fig = plt.figure()
ax = fig.gca(projection='3d')
Times,x_axis = meshgrid(t,range(N))
#make the waterfall plot
surf = ax.plot_wireframe(x_axis,Times,ez,rstride=1000, cstride=12)
ax.set_zlim3d(-ez.max(), 2*ez.max()*2)

#dress up the plot
xlabel("Space [index]")
ylabel("Time [index]")
ax.set_zlabel("E-Field Amp. [arb]")
title("")

```

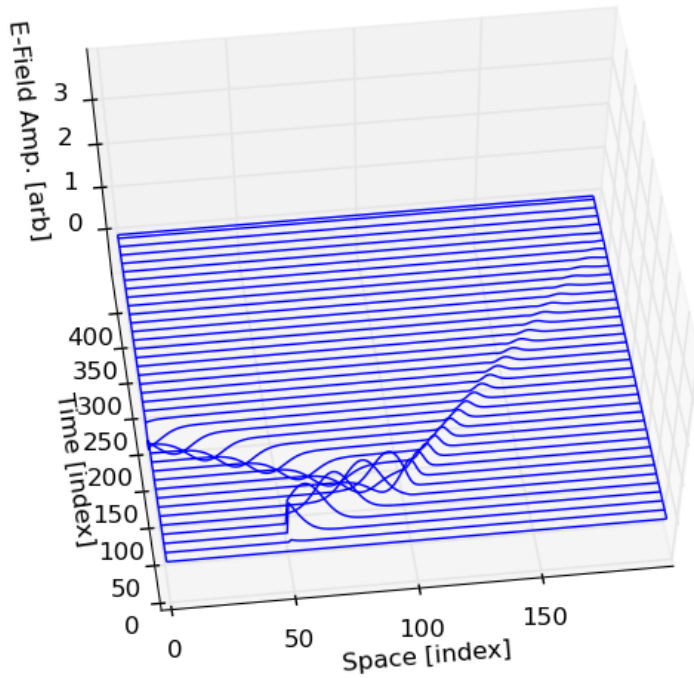

Figure 6: Plot with lossy dielectic.

```
show(True)
```

B Complete Codes

B.1 Additive Source

```c
/*
author: John Nehls
updated: 5/22/14

Bare-bones, homogeneous, 1D FDTD simulation with a hard source. Grids are defined as in Chapter 3 of Understanding the Finite-Difference TIme-Domain Method by John B. Shneider.
*/

/* grid: ez[0]   hy[0]   ez[1]   hy[1]  ... ez[N-1]   hy[N-1]  */

/* the Courant number is unity (ie, c*dt = dx) */

#include <stdio.h>
#include <math.h>                /* must compile with flags lm to use */
#define  N 200                   /* size of ez and hy arrays */

int main(){

  double imp0 = 377.0;          /* impedence of freespace */
  int maxT = 250;                /* total simulation time */
  int n, curT;                   /* cannot declare variable in loop in c */

  /* initialize grids */
  double ez[N] = {0.0} , hy [N] = {0.0};

  /* for outputting data */
  FILE *fp;
  fp=fopen("data.txt","w");
  /* output paramters to the first line */
  fprintf(fp, "%d %s %d %s %f %c ", N, " ",  maxT,  " ",  imp0, '\n');


  /* Step in time */
  for ( curT = 0; curT < maxT; curT++){

    /* Magnetic Field */
    /* update H-field -- doesn't evaluate last node, it remains 0 (PMC)*/
    for (n = 0; n < N-1; n++){
      hy[n] += (ez[n+1] - ez[n]) / imp0;
    }

    /* Electric Field */
    /* update E-field -- doesn't evaluate first node, it remains 0 (PEC)*/
    for (n = 1; n < N; n++){
      ez[n] += (hy[n] - hy[n-1]) * imp0;
    }

    /* CHOOSE SOURCE -- comment out unused source */
    /* hardwire a source node */
    //ez[0] = exp(-(curT - 30.0) * (curT - 30.0) / 100.0);
    /* additive source at node 50 */
    ez[50] += exp(-(curT - 30.) * (curT - 30.) / 100.);

    /* write data */
    for (n = 0; n < N; n++){
      fprintf(fp, "%d %s %f %s %f %c", curT, " ", ez[n], " ", hy[n], '\n');
    }
  }
  /* close the writer */
  int fclose(fp);
  return 0;
}
```

B.2 ABC

```c
/*
author: John Nehls
updated: 5/22/14

Bare-bones, homogeneous, 1D FDTD simulation with a hard source. Grids are defined as in Chapter 3 of Understanding the Finite-Difference TIme-Domain Method by John B. Shneider.
*/

/* grid: ez[0]   hy[0]   ez[1]   hy[1]  ... ez[N-1]   hy[N-1]  */

/* the Courant number is unity (ie, c*dt = dx) */

#include <stdio.h>
#include <math.h>                /* must compile with flags lm to use */
#define  N 200                   /* size of ez and hy arrays */

int main(){

  double imp0 = 377.0;          /* impedence of freespace */
  int maxT = 450;                /* total simulation time */
  int n;                         /* cannot declare variable in loop in c */
  double ez[N] = {0.0} , hy [N] = {0.0}; /* initialize grids */
  int curT;                      /* current time */

  /* for outputting data */
  FILE *fp;
  fp=fopen("data.txt","w");
  /* output paramters to the first line */
  fprintf(fp, "%d %s %d %s %f %c ", N, " ",  maxT,  " ",  imp0, '\n');


  /* Step in time */
  for ( curT = 0; curT < maxT; curT++){

    /* Magnetic Field */
    /* simple absorbing boundary conditions */
    hy[N-1] = hy[N-2];

    /* update H-field -- doesn't evaluate last node, it remains 0 (PMC)*/
    for (n = 0; n < N-1; n++){
      hy[n] += (ez[n+1] - ez[n]) / imp0;
    }

    /* Electric Field */
    /* simple absorbing boundary conditions */
    ez[0] = ez[1];
    /* update E-field -- doesn't evaluate first node, it remains 0 (PEC)*/
    for (n = 1; n < N; n++){
      ez[n] += (hy[n] - hy[n-1]) * imp0;
    }

    /* CHOOSE SOURCE -- comment out unused source */
    /* hardwire a source node */
    //ez[0] = exp(-(curT - 30.0) * (curT - 30.0) / 100.0);
    /* additive source at node 50 */
    ez[50] += exp(-(curT - 30.) * (curT - 30.) / 100.);

    /* write data */
    for (n = 0; n < N; n++){
      fprintf(fp, "%d %s %f %s %f %c", curT, " ", ez[n], " ", hy[n], '\n');
    }
  }
  int fclose(fp);
  return 0;
}
```

B.3 TFSF

```c
/*
author: John Nehls
updated: 5/22/14

1D FDTD simulation with simple a ABC and a TFSF boundary between hy[49] and ez[50]. Grids are defined as in Chapter 3 of Understanding the Finite-Difference TIme-Domain Method by John B. Shneider.
*/

/* grid: ez[0]   hy[0]   ez[1]   hy[1]  ... ez[N-1]   hy[N-1]  */

/* the Courant number is unity (ie, c*dt = dx) */

#include <stdio.h>
#include <math.h>
#define N 200                    /* size of ez and hy arrays */

int main(){

  double imp0 = 377.0;          /* impedence of freespace */
  int maxT = 450;                /* total simulation time */
  int n, curT;                   /* cannot declare variable in loop in c*/

  double ez[N] = {0.0} , hy [N] = {0.0}; /* initialize grids */

  /* for outputting data */
  FILE *fp;
  fp=fopen("data.txt","w");
  /* output paramters to the first line */
  fprintf(fp, "%d %s %d %s %f %c ", N, " ",  maxT,  " ",  imp0, '\n');

  /* Step in time */
  for ( curT = 0; curT < maxT; curT++){

    /* Magnetic Field */
    /* simple absorbing boundary condition */
    hy[N-1] = hy[N-2];
    /* update H-field -- doesn't evaluate last node, it remains 0 (PMC)*/
    for (n = 0; n < N-1; n++){
      hy[n] += (ez[n+1] - ez[n]) / imp0;
    }
    /* TFSF correction adjacent to the boudary --also the source */
    hy[49] -= exp(-(curT - 30.) * (curT - 30.)/100.) / imp0;


    /* Electric Field */
    /* simple absorbing boundary condition */
    ez[0] = ez[1];
    /* update E-field -- doesn't evaluate first node, it remains 0 (PEC)*/
    for (n = 1; n < N; n++){
      ez[n] += (hy[n] - hy[n-1]) * imp0;
    }
    /* TFSF correction adjacent to the boudary --also the source */
    ez[50] += exp(-(curT + 0.5 - (-0.5) - 30.) * (curT + 0.5 - (-0.5) - 30.)/100.);


    /* write data */
    for (n = 0; n < N; n++){
      fprintf(fp, "%d %s %f %s %f %c", curT, " ", ez[n], " ", hy[n], '\n');
    }
  }
  /* close the writer */
  int fclose(fp);
  return 0;
}
```

B.4 Inhomogeneous Medium

```c
/*
author: John Nehls
updated: 5/22/14

1D FDTD simulation with simple a ABC and a TFSF boundary between hy[49] and ez[50]. Grids are defined as in Chapter 3 of Understanding the Finite-Difference TIme-Domain Method by John B. Shneider.
*/

/* grid: ez[0]   hy[0]   ez[1]   hy[1]  ... ez[N-1]   hy[N-1]  */

/* the Courant number is unity (ie, c*dt = dx) */

#include <stdio.h>
#include <math.h>                /* must use lm flags while compiling */
#define N 200                    /* size of ez and hy arrays */


int main(){

  double imp0 = 377.0;          /* impedence of freespace */
  int maxT = 450;                /* total simulation time */
  int curT;                      /* current time */
  int n;                         /* cannot declare variable in loop in c */

  double ez[N] = {0.0} , hy [N] = {0.0}, epsR[N]= {0.0}; /* initialize grids */
  /* set relative permittivity */
  for (n = 0; n < N; n++)
    if (n < 100)
      epsR[n] = 1.0;
    else
      epsR[n] = 9.0;

  /* for outputting data */
  FILE *fp;
  fp=fopen("data.txt","w");
  /* output paramters to the first line */
  fprintf(fp, "%d %s %d %s %f %c ", N, " ",  maxT,  " ",  imp0, '\n');

  /* Step in time */
  for ( curT = 0; curT < maxT; curT++){

    /* Magnetic Field */
    /* simple absorbing boundary condition */
    hy[N-1] = hy[N-2];
    /* update H-field -- doesn't evaluate last node, it remains 0 (PMC)*/
    for (n = 0; n < N-1; n++){
      hy[n] += (ez[n+1] - ez[n]) / imp0;
    }
    /* TFSF correction adjacent to the boudary --also the source */
    hy[49] -= exp(-(curT - 30.) * (curT - 30.)/100.) / imp0;


    /* Electric Field */
    /* simple absorbing boundary condition */
    ez[0] = ez[1];
    /* update E-field -- doesn't evaluate first node, it remains 0 (PEC)*/
    for (n = 1; n < N; n++){
      ez[n] += (hy[n] - hy[n-1]) * imp0 / epsR[n];
    }
    /* TFSF correction adjacent to the boudary --also the source */
    ez[50] += exp(-(curT + 0.5 - (-0.5) - 30.) * (curT + 0.5 - (-0.5) - 30.)/100.);

    /* write data */
    for (n = 0; n < N; n++){
      fprintf(fp, "%d %s %f %s %f %c", curT, " ", ez[n], " ", hy[n], '\n');
    }
  }
  int fclose(fp);
  return 0;
}
```

B.5 Lossy Material

B.5.1 Simple Lossy

```c
/*
author: John Nehls
updated: 5/22/14

1D FDTD simulation with simple a ABC and a TFSF boundary between hy[49] and ez[50]. Grids are defined as in Chapter 3 of Understanding the Finite-Difference TIme-Domain Method by John B. Shneider.
*/

/* grid: ez[0]   hy[0]   ez[1]   hy[1]  ... ez[N-1]   hy[N-1]  */

/* the Courant number is unity (ie, c*dt = dx) */


#include <stdio.h>
#include <math.h>                /* must compile with flags lm to use */
#define  N 200                   /* size of ez and hy arrays */
#define loss 0.01                /* loss in the lossy area */

int main(){

  double imp0 = 377.0;          /* impedence of freespace */
  int maxT = 450;                /* total simulation time */
  int n, curT;                   /* cannot declare variable in loop in c */

  /* initialize grids */
  double ez[N] = {0.0} , hy [N] = {0.0}, ceze[N] = {0.0}, cezh[N] = {0.0};

  /* set coefficients*/
  for (n = 0; n < N; n++)
    if (n < 100) { /* free space */
      ceze[n] = 1.0;
      cezh[n] = imp0;
    } else { /* lossy dielectric */
      ceze[n] = (1.0 - loss) / (1.0 + loss);
      cezh[n] = imp0 / 9.0 / (1.0 + loss);
    }

  /* for outputting data */
  FILE *fp;
  fp=fopen("data.txt","w");
  /* output paramters to the first line */
  fprintf(fp, "%d %s %d %s %f %c ", N, " ",  maxT,  " ",  imp0, '\n');

  /* Step in time */
  for ( curT = 0; curT < maxT; curT++){

    /* Magnetic Field */
    /* simple absorbing boundary condition */
    hy[N-1] = hy[N-2];
    /* update H-field -- doesn't evaluate last node, it remains 0 (PMC)*/
    for (n = 0; n < N-1; n++){
      hy[n] += (ez[n+1] - ez[n]) / imp0;
    }
    /* TFSF correction adjacent to the boudary --also the source */
    hy[49] -= exp(-(curT - 30.) * (curT - 30.)/100.) / imp0;


    /* Electric Field */
    /* simple absorbing boundary condition */
    ez[0] = ez[1];
    /* update E-field -- doesn't evaluate first node, it remains 0 (PEC)*/
    for (n = 1; n < N; n++){
      ez[n] = ceze[n]* ez[n] + cezh[n]*(hy[n] - hy[n-1]);
    }
    /* TFSF correction adjacent to the boudary --also the source */
    ez[50] += exp(-(curT + 0.5 - (-0.5) - 30.) * (curT + 0.5 - (-0.5) - 30.)/100.);


```

```
    /* write data */
    for (n = 0; n < N; n++){
      fprintf(fp, "%d %s %f %s %f %c", curT, " ", ez[n], " ", hy[n], '\n');
    }
  }
  /* close the writer */
  int fclose(fp);
  return 0;
}
```

B.5.2 Impedance Matched

```c
/*
author: John Nehls
updated: 5/22/14

1D FDTD simulation with simple a ABC and a TFSF boundary between hy[49] and ez[50]. Grids are defined as in Chapter 3 of Understanding the Finite-Difference TIme-Domain Method by John B. Shneider.
*/

/* grid: ez[0]   hy[0]   ez[1]   hy[1]  ... ez[N-1]   hy[N-1]  */

/* the Courant number is unity (ie, c*dt = dx) */

#include <stdio.h>
#include <math.h>
#define N 200                    /* size of ez and hy arrays */
#define loss 0.02                /* loss in the simulation */
#define lossBoundary 180         /* index of the boundary */

int main(){

  double imp0 = 377.0;          /* impedence of freespace */
  int maxT = 450;                /* total simulation time */
  int n, curT;                   /* cannot declare variable in loop in c*/

  /* initialize grids */
  double ez[N] = {0.0} , hy [N] = {0.0}, ceze[N] = {0.0}, cezh[N] = {0.0}, chyh[N] = {0.0}, chye[N] = {0.0};

  /* set coefficients*/
  /* set electric-field update coefficients */
  for (n = 0; n < N; n++){
    if (n < 100) {
      ceze[n] = 1.0;
      cezh[n] = imp0;
    } else if (n < lossBoundary) {
      ceze[n] = 1.0;
      cezh[n] = imp0 / 9.0;
    } else {
      ceze[n] = (1.0 - loss) / (1.0 + loss);
      cezh[n] = imp0 / 9.0 / (1.0 + loss);
    }
  }

  /* set magnetic-field update coefficients */
  for (n = 0; n < N; n++){
    if ( n < lossBoundary){
      chyh[n] = 1.0;
      chye[n] = 1.0 / imp0;
    }
    else{
      chyh[n] = (1 - loss)/(1 + loss);
      chye[n] = 1.0/ imp0 / (1.0 + loss);
    }
  }

  /* for outputting data */
  FILE *fp;
  fp=fopen("data.txt","w");
  /* output paramters to the first line */
  fprintf(fp, "%d %s %d %s %f %c ", N, " ",  maxT,  " ",  imp0, '\n');

  /* Step in time */
  for ( curT = 0; curT < maxT; curT++){

    /* Magnetic Field */

    /* update H-field -- doesn't evaluate last node, it remains 0 (PMC)*/
    for (n = 0; n < N-1; n++){
      hy[n] = chyh[n]*hy[n] + chye[n]*(ez[n+1] - ez[n]);
    }
```

```
    /* TFSF correction adjacent to the boudary --also the source */
    hy[49] -= exp(-(curT - 30.) * (curT - 30.)/100.) / imp0;


    /* Electric Field */
    /* simple absorbing boundary condition */
    ez[0] = ez[1];
    /* update E-field -- doesn't evaluate first node, it remains 0 (PEC)*/
    for (n = 1; n < N; n++){
      ez[n] = ceze[n]* ez[n] + cezh[n]*(hy[n] - hy[n-1]);
    }
    /* TFSF correction adjacent to the boudary --also the source */
    ez[50] += exp(-(curT + 0.5 - (-0.5) - 30.) * (curT + 0.5 - (-0.5) - 30.)/100.);

    /* write data */
    for (n = 0; n < N; n++){
      fprintf(fp, "%d %s %f %s %f %c", curT, " ", ez[n], " ", hy[n], '\n');
    }
  }
  /* close the writer */
  int fclose(fp);
  return 0;
}
```